Name: _____

Student Number: _____

Signature: _____

Hand in this test once you are finished.

Nelson Mandela Metropolitan University
for tomorrow

NELSON MANDELA METROPOLITAN UNIVERSITY
SUMMERSTRAND CAMPUS (SOUTH)

SEMESTER TEST QUESTION PAPER

12 May 2017

SUBJECT: **Advanced Programming 3.1**

MODULE CODE: **WRAP301/WRPV301**

TIME : 2½ **hours**

MARKS: **75**

EXAMINERS:

1. **Dr. D. Vogts**

MODERATORS:

1. **Prof. C. B. Cilliers**

SPECIAL INSTRUCTIONS

1. Answer **ALL** questions.
2. This question paper consists of 2 sections:
   - Section A: 25 marks **written questions.**
   - Section B: 50 marks Java **practical question.**
3. Section A must be completed and handed in before Section B may be started.
4. No permitted literature for Section A.
5. Permitted materials for Section B:
   - JDK online documentation (http://docs.oracle.com/javase/8/docs/api/ or on C:).
   - Bound lecture notes.
   - Online lecture notes (//cs2/courses/wrap301/WRAP301 Notes.pdf).
   - Clean Java text book (does not have to be prescribed one, but may not be written in).
   - Internet access is allowed, **but** file repositories and online chatting are expressly forbidden. Forums, developer sites, API documentation, Google, etc. are allowed (read only). Misuse of the internet during the test is a punishable offence – if unsure, ask!
6. Your solutions for the questions in this paper **must** be your own original work.
7. **Each of your source files (*.java) must include your name and student number at the top of each listing (as a comment).**
8. You must place a copy of your code on the T: drive in the sub-folder allocated to you.

Section A: Written Questions (Closed Book) **[25]**

*Note: You may **not** use your lecture notes or text book when answering these questions!*

1.1) What is *regression testing* and why is it important? (2)

1.2) In Assignment 6 you write code to load and save an object from and to an XML file using reflection. In Assignment 7, you were required to use unit tests to prove that the load and save methods worked correctly. Briefly describe the *strategy* you followed and *motivate* why you did it in this manner. (6)

1.3) What is the difference between *black box* and *glass box* testing? (2)

2) JavaFX makes use of Stages, Scenes and Nodes. Briefly discuss each in turn. (6)

3) What is *Reflection*? What does it allow you to do? (3)

4) Briefly discuss how the Model-View-Controller pattern works. (6)

Once completed, hand in your answers for Section A to receive Section B.

Section B: Programming Question **[50]**

Username: **Password:**

Note: You may use your lecture notes, text book and JavaDocs and Internet (read conditions on front page) when answering these questions!

1) Reflection *must* be used when implementing this task! (19)

Prior to JavaFX, a *property* `P` of type `T` was defined by the presence of one or both of the methods named `getP` and `setP`. The signatures for these methods is as follows:

- `void setP(T value)` – i.e. returns nothing, but accepts a single parameter of type `T`
- `T getP()` – i.e. returns a value of type `T`, but accepts no parameters

The presence of the `setP` method indicates *write access* to the property `P` and the presence of the `getP` method indicates *read access* to the property `P`.

For example, if a class had the following methods:

```
Integer getWidth()                      Class<?> getClass()
void setWidth(Integer value)
                                        int getCount()
String getName()
void setName(String value)              float getArea(float x, float y)
```

then the class has four *properties*, namely:

- `width`, type `Integer`, read and write access
- `name`, type `String`, read and write access
- `class`, type `Class<?>`, read only access
- `count`, type `int`, read only access

`Area` is not a property since the `getArea` method has two parameters.

For this task, you are to implement the class called `ReflectiveProperty` as described below. The purpose of the class is to provide a single object that represents a property of a specific object. When the reflective property object is created, the object being inspected, the name of the property and the type of the property are passed through as parameters. The property can be readable (if it has the appropriate get method) and/or writeable (if it has the appropriate set method). The value of the property can then be get or set in a simple, uniform manner.

Typical usage of a reflective property is shown in the code snippet below:

```
// create a reflective property called 'name' that uses the String getName()
// and void setName(String name) methods to get and set the values (using reflection).
ReflectiveProperty<String> name = new ReflectiveProperty<>(person, "name", String.class);

// set the name – equivalent to person.setName("Bob");
name.set("Bob");

// get the value – equivalent to person.getName();
System.out.println(name.get());
```

n	Description	Marks
M_1	`private Method getGetter(Object object, String propertyName, Class propertyType)`	(7)
	Returns the method object defined for the given object with the name "get" + property name (watch capitalisation) has no parameters and that returns a value of type property type. If no such method exists, then null is returned instead.	
M_2	`private Method getSetter(Object object, String propertyName, Class propertyType)`	(4)

	Returns the method object defined for the given object with the name "set" + property name (watch capitalisation) that accepts a single parameter of type property type. If no such method exists, then null is returned instead.	
M_3	`public ReflectiveProperty(Object object, String propertyName, Class propertyType)`	(2)
	Constructor that creates a reflective property for the given object. Property name specifies the name of the property, i.e. *P* discussed above. Property type specifies the type of the parameter used by the set method and the return type of the get method. The constructor must obtain Method objects for the getter and setter methods of the object that can be invoked to get or set the property.	
M_4	`public void set(T value)`	(2)
	Sets the object's property to the given value if it is writeable.	
M_5	`public T get()`	(2)
	Returns the object's property's value if the property is readable. If not, then the method returns null.	
M_6	`public boolean isReadable()`	(1)
	Returns true if the property is readable.	
M_7	`public boolean isWriteable()`	(1)
	Returns true if the property is writeable.	

Use the test classes provided in the test folder to check that your class is functioning correctly. Note that it should not be necessary to modify these classes at all, except for importing the package in which you defined the reflective property class.

2) The MVC pattern *must* be used when implementing this task! (31)

For this task, you are to connect to the SQL Server with the following credentials:

Host machine and instance: OPENBOX\WRR
Database name: WRAP301XM1
Username: T2017
Password: 1

and interact with the Customers table. The screenshot below shows what the first five rows of the table currently look like:

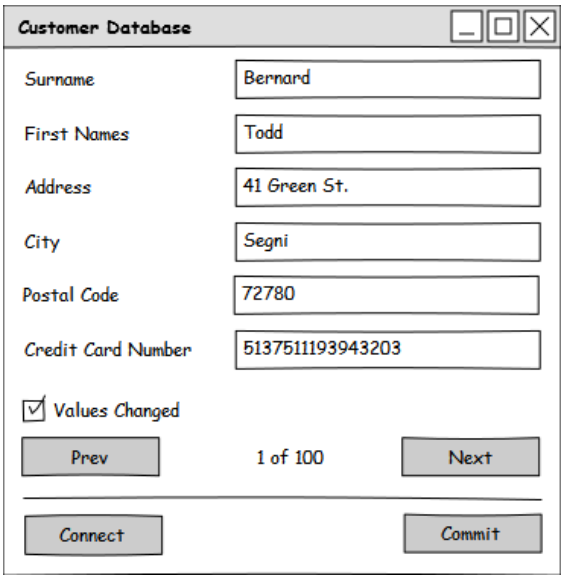
pID	surname	firstNames	address	city	postalCo...	creditCardNumber
1	Bernard	Todd	941-580 Enim St.	Segni	72780	5137511193943203
2	Atkins	Xerxes	Ap #424-543 Pellentesque Street	Lombardsijde	24-899	5116004824643021
3	Barber	Adara	4208 Pellentesque, Rd.	Argyle	95746	372901926232506
4	Randolph	Quintessa	806-9060 Magna. Rd.	Chestermere	11777	379681904327121
5	Stephenson	Maris	774-5508 At St.	Lummen	49343	374212845733855

You are to write a JavaFX application that has a layout similar to that shown in the mock-up screen below and provide the specified functionality.

When the user clicks on the *connect* button, the application connects to the Customer database using the credentials given above. It then selects all the rows from the customer table and reads each of these into a customer *object*.

A customer object has an original value for each field, as well as an editable version which the user can change in the user interface. If the original value and the editable value for a field are not the same, then the customer's details has been *modified* (it has changed). A customer's fields may be *reverted* back to the original values by calling the *revert* method. Finally, a customer object may *update* its row in the Customer database by calling the *update* method. Following an update, the original and editable field values are the same. The primary key for a customer, *pID*, must not be editable.

When the user clicks on the *commit* button, any modified customer objects update their rows in the Customer database.



The customer objects are displayed one at a time in the UI and the text fields are bound (using JavaFX's property binding) to the current customer object's editable fields. As each text field changes, so too does the editable field it is bound too. If the original and editable versions of a field differ, then the *values changed* check box must automatically change as well. If the *values changed* check box is unchecked by the user, then the changes made must be *reverted*.

The customer objects may be navigated using the *prev* and *next* buttons. The label displaying "X of Y" must automatically change as navigation occurs using property binding.

Once completed:

- **Remember to put your name and student number at the top of each *.java file.**
- **NO PRINTING!**
- **SAVE your work to the T:**
- **Remember that if you do not save all your work to the T:, there is nothing to mark, i.e. expect 0!**
- **You may not leave the venue until the last 15 minutes of the test.**